Hello, mdsh

First we decide what to say and who to say it to. `export` makes `GREETING` visible to any sub-env we start later; `NAME` is a plain shell variable, so it stays in this shell.

```sh {#setup}
export GREETING=hello
NAME=world
echo "$GREETING, $NAME"
```

```
hello, world
```

Blocks without `env=` all run in the same shell, like lines of one script, so this block still sees `NAME`. It counts the letters in it: `printf` avoids a trailing newline, `wc -c` counts bytes, and `tr` strips the padding some `wc`s add.

```sh {#count}
printf '%s' "$NAME" | wc -c | tr -d ' '
```

```
5
```

A sub-env is a separate shell that only receives the exported variables named in `share=`. `GREETING` comes through. `NAME` doesn't, because it was never exported, so `${NAME:-unset}` falls back to `unset`.

```sh {#isolated env=sub share=GREETING}
echo "GREETING=$GREETING NAME=${NAME:-unset}"
```

```
GREETING=hello NAME=unset
```

A block can also write markdown. `out=markdown` puts its output into the document as-is, and `md_table` turns tab-separated lines (header first) into a table.

```sh {#table out=markdown}
printf 'variable\tvalue\nGREETING\t%s\nNAME\t%s\n' "$GREETING" "$NAME" | md_table
```

variable	value
GREETING	hello
NAME	world

A block's output can also be an image. With `out=image`, the block prints image bytes (PNG, JPEG, GIF or SVG), and mdsh saves them next to this file in `hello.md.assets/` and links them here. This block draws an SVG banner with plain `printf`: the bar is 12 pixels wide per letter of the greeting, using shell arithmetic `$(( ))`.

```sh {#banner out=image}
w=$(( ${#GREETING} * 12 ))
printf '<svg xmlns="http://www.w3.org/2000/svg" width="240" height="40">'
printf '<rect width="%d" height="40" rx="6" fill="#4f7cff"/>' "$w"
printf '<text x="%d" y="26" font-family="sans-serif" font-size="16" fill="#888">%s, %s
</text>' "$(( w + 8 ))" "$GREETING" "$NAME"
printf '</svg>\n'
```

[image: banner]

To take a picture of plain text output instead, use `out=screenshot`: whatever the block prints is drawn as a terminal-style image. It's an SVG by default; ending the path in `.png` asks for a PNG. Here `printf '%-10s'` pads each name to 10 columns so the values line up.

```
sh {#screenshot out=screenshot:hello.md.assets/screenshot.png}
printf '%-10s %s\n' GREETING "$GREETING" NAME "$NAME"
```



Some parts are only for debugging. Everything between the `mdsh:if` and `mdsh:end` comments below is printed only when `DEBUG` is set (try `DEBUG=1 ./hello.md`), and `run={{$DEBUG}}` keeps the block from running otherwise. Its stderr is shown too, because of `show-stderr`; normally stderr only appears when a block fails.

Prose can quote what a block left behind: `setup` set the greeting to **hello** and greeted **world**. It can also quote the environment mdsh was started in: this ran as **j**.